

```

# Import necessary libraries
from collections import deque

# Define the dimensions of the puzzle
N = 3

# Structure to store a state of the puzzle
class PuzzleState:
    def __init__(self, board, x, y, depth):
        self.board = board
        self.x = x
        self.y = y
        self.depth = depth

# Possible moves: Left, Right, Up, Down
row = [0, 0, -1, 1]
col = [-1, 1, 0, 0]

# Function to check if a given state is the goal state
def is_goal_state(board):
    goal = [[1,2,3], [8,0,4], [7,6,5]]
    return board == goal

# Function to check if a move is valid
def is_valid(x, y):
    return 0 <= x < N and 0 <= y < N

# Function to print the board
def print_board(board):
    for row in board:
        print(' '.join(map(str, row)))
    print("-----")

# Depth-First Search to solve the 8-puzzle problem
def solve_puzzle_dfs(start, x, y):
    stack = []
    visited = set()

    stack.append(PuzzleState(start, x, y, 0))
    visited.add(tuple(map(tuple, start)))

    while stack:
        curr = stack.pop()

        # Print the current board
        print(f'Depth: {curr.depth}')
        print_board(curr.board)

        # Check if goal state is reached
        if is_goal_state(curr.board):
            print(f'Goal state reached at depth {curr.depth}')
            return

        # Explore possible moves
        for i in range(4):
            new_x = curr.x + row[i]
            new_y = curr.y + col[i]

            if is_valid(new_x, new_y):
                new_board = [row[:] for row in curr.board]
                # Swap the tiles
                new_board[curr.x][curr.y], new_board[new_x][new_y] = new_board[new_x][new_y], new_board[curr.x][curr.y]

                # If this state has not been visited before, push to stack
                board_tuple = tuple(map(tuple, new_board))
                if board_tuple not in visited:
                    visited.add(board_tuple)
                    stack.append(PuzzleState(new_board, new_x, new_y, curr.depth+1))

    print('No solution found (DFS Brute Force reached depth limit)')

# Driver Code
if __name__ == '__main__':
    start = [[2,8,3], [1,6,4], [7,0,5]]
    x, y = 1, 1

```

```
print('Initial State:')
print_board(start)

solve_puzzle_dfs(start, x, y)
```

```
-----
Depth: 307
1 4 5
7 3 6
0 2 8
```

```
-----
Depth: 308
1 4 6
7 3 5
0 2 8
```

```
-----
Depth: 309
1 6 4
7 3 5
0 2 8
```

```
-----
Depth: 310
1 3 4
7 6 5
0 2 8
```

```
-----
Depth: 311
1 3 4
7 2 5
0 6 8
```

```
-----
Depth: 312
1 3 4
7 2 5
0 8 6
```

```
-----
Depth: 313
1 3 4
7 2 6
0 8 5
```

```
-----
Depth: 314
1 3 6
7 2 4
0 8 5
```

```
-----
Depth: 315
1 6 3
7 2 4
0 8 5
```

```
-----
Depth: 316
1 2 3
7 6 4
0 8 5
```

```
-----
Depth: 317
1 2 3
7 8 4
0 6 5
```

```
-----
Depth: 318
1 2 3
7 8 4
```

